

```
// =====
// ==                      CSVReader.h                      ==
// =====

#pragma once

#include "OrderBookEntry.h"
#include <vector>
#include <string>

class CSVReader
{
public:
    /** Just a constructor. */
    CSVReader();

    /** Read CSV file and turn into list of orders. */
    static std::vector<OrderBookEntry> readCSV(std::string csvFile);
    /** Split string into pieces based on separator. */
    static std::vector<std::string> tokenise(std::string csvLine, char separator);

    /** Convert simple strings to OrderBookEntry object. */
    static OrderBookEntry stringsToOBE(std::string price,
                                        std::string amount,
                                        std::string timestamp,
                                        std::string product,
                                        OrderBookType OrderBookType);

private:
    /** Convert list of strings to OrderBookEntry object. */
    static OrderBookEntry stringsToOBE(std::vector<std::string> strings);
};
```

```
// =====
// ==                      Candlestick.h                      ==
// =====

#pragma once
#include <string>
#include "OrderBookEntry.h"

class Candlestick {
public:
    /** Make new candle object with all the price info. */
    Candlestick(std::string timestamp,
                std::string product,
                OrderBookType orderType,
                double open,
                double high,
                double low,
                double close);

    /** Get price when market open. */
    double getOpen() const;
    /** Get highest price. */
    double getHigh() const;
    /** Get lowest price. */
    double getLow() const;
    /** Get price when market close. */
    double getClose() const;
    /** Get which product this is for. */
    std::string getProduct() const;
    /** Get when this candle happen. */
    std::string getTime() const;
    /** Get if this is ask or bid. */
    OrderBookType getOrderType() const;

private:
    double open;
    double high;
    double low;
    double close;
    std::string timestamp;
    std::string product;
    OrderBookType orderType;
};
```

```

// =====
// ==                  CandlestickCalculator.h                  ==
// =====

#pragma once
#include <vector>
#include <string>
#include "OrderBookEntry.h"
#include "Candlestick.h"

enum class TimeGroup {Daily, Monthly, Yearly};

class CandlestickCalculator {
public:
    /** Calculate candlesticks from a bunch of orders. Can group by day, month or year. *
    /
    static std::vector<Candlestick> calculate(const std::vector<OrderBookEntry>& orders,
        std::string product,
        OrderBookType orderType,
        TimeGroup timeGroup,
        bool includeBoth = false);
private:

};

```

```
// =====
// ==                      Candlestickplotter.h                      ==
// =====

#pragma once
#include <vector>
#include <string>
#include "Candlestick.h"

/**
 *THIS COMMENT WAS AI GENERATED FOR ME TO HAVE A CLEARLY STATED IDEA ON WHAT I WANTED TO
DO
 * CandlestickPlotter renders a vector of Candlestick objects as an ASCII/ANSI
 * chart in the terminal. Green (â\226\210) = close >= open, red (â\226\221) = close < op
en.
 * Wicks drawn as â\224\202. Stateless utility, hence static.
 */
class CandlestickPlotter {
public:
    /** Draws the chart. Shows at most maxCandles (the most recent ones). */
    static void plot(const std::vector<Candlestick>& candles,
                    int height = 20,
                    int maxCandles = 18);

private:
    /** This one change price to row number. Low price at bottom, high at top. */
    static int priceToRow(double p, double lo, double hi, int height);

    /** Cut date string so it can fit under candle. */
    static std::string dateLabel(const std::string& time);

    /** Make price look nice, no matter if it big or small. */
    static std::string formatPrice(double v);
};
```

```

// =====
// ==                                MerkelMain.h                                ==
// =====

#pragma once

#include <vector>
#include <string>
#include "OrderBookEntry.h"
#include "OrderBook.h"
#include "Candlestick.h"
#include "CandlestickCalculator.h"
#include "Candlestickplotter.h"
#include "UserWallet.h"
#include "UserManager.h"
#include "TransactionLog.h"
#include "Simulator.h"

class MerkelMain
{
public:
    MerkelMain();
    /** Start everything. */
    void init();
private:
    // ---- menus ----
    /** Show menu for login. */
    void printLoginMenu();                // shown when logged out
    /** Show main menu for logged in user. */
    void printMainMenu();                // shown when logged in
    /** Wait and get what user type. Make sure it correct. */
    int  getUserOption(int maxOption);    // validated 1..maxOption, loops on bad
input
    /** Decide what to do with login menu choice. */
    void processLoginOption(int option, bool& running);
    /** Decide what to do with main menu choice. */
    void processMainOption(int option);

    // ---- help (context specific) ----
    /** Explain how login work. */
    void printLoginHelp();
    /** Explain how main menu work. */
    void printMainHelp();

    // ---- Task 2 auth handlers ----
    /** Create new account. */
    void doRegister();
    /** Log into existing account. */
    void doLogin();
    /** Change password if forgot. */
    void doResetPassword();

    // ---- Task 1 ----
    /** Ask user which product and time they want to see. */
    std::vector<Candlestick> getCandlesFromUser(bool askType = true); // shared prompt (
product/type/group)
    /** Show candle table on screen. */
    void printCandlestickData();        // OHLC table
    /** Draw the chart in terminal. */
    void doPlotChart();                // ASCII/ANSI chart

    // ---- trading handlers ----
    /** Run the auto trader. */
    void doSimulation();                // Task 4
    /** Jump to next time period. */
    void gotoNextTimeframe();          // match demo

    // ---- state ----
    std::string currentTime;
    std::string currentUser;           // empty = logged out

```

```
// ---- members (order matters: userWallet before its referrers) ----
OrderBook      orderBook{"./20200317.csv"};
UserWallet     userWallet{"data/USERS_WALLET.CSV"};
UserManager    userManager{userWallet};
TransactionLog txLog{"data/USERS_TRADING.CSV"};
Simulator      simulator{orderBook, userWallet, txLog};
};
```

```
// =====
// ==                                OrderBook.h                                ==
// =====

#pragma once
#include "OrderBookEntry.h"
#include "CSVReader.h"
#include <string>
#include <vector>

class OrderBook
{
public:
    /** Load all the order data from CSV file. */
    OrderBook(std::string filename);
    /** Tell me what coins/products we have. */
    std::vector<std::string> getKnownProducts();
    /** Find orders that match what you looking for. */
    std::vector<OrderBookEntry> getOrders(OrderBookType type,
                                           std::string product,
                                           std::string timestamp);

    /** Give the first timestamp in the book. */
    std::string getEarliestTime();
    /** Find what time come after this one. If end, go back to start. */
    std::string getNextTime(std::string timestamp);

    /** Put new order into the book. */
    void insertOrder(OrderBookEntry& order);

    /** Match buy and sell orders. */
    std::vector<OrderBookEntry> matchAsksToBids(std::string product, std::string time
stamp);

    /** Find highest price in this list of orders. */
    static double getHighPrice(std::vector<OrderBookEntry>& orders);
    /** Find lowest price in this list of orders. */
    static double getLowPrice(std::vector<OrderBookEntry>& orders);

    /** Get the whole list of orders. */
    const std::vector<OrderBookEntry>& getOrderBook() const;

    //my code ends here

private:
    std::vector<OrderBookEntry> orders;
};
```

```
// =====
// ==                OrderBookEntry.h                ==
// =====

#pragma once

#include <string>

enum class OrderBookType{bid, ask, unknown, asksale, bidsale};

class OrderBookEntry
{
    public:

    /** Make new order entry. */
    OrderBookEntry( double _price,
                    double _amount,
                    std::string _timestamp,
                    std::string _product,
                    OrderBookType _orderType,
                    std::string username = "dataset");

    /** Convert string to order type like bid or ask. */
    static OrderBookType stringToOrderBookType(std::string s);

    /** Compare two orders by time. */
    static bool compareByTimestamp(OrderBookEntry& e1, OrderBookEntry& e2)
    {
        return e1.timestamp < e2.timestamp;
    }

    /** Compare two orders by price, low to high. */
    static bool compareByPriceAsc(OrderBookEntry& e1, OrderBookEntry& e2)
    {
        return e1.price < e2.price;
    }

    /** Compare two orders by price, high to low. */
    static bool compareByPriceDesc(OrderBookEntry& e1, OrderBookEntry& e2)
    {
        return e1.price > e2.price;
    }

    double price;
    double amount;
    std::string timestamp;
    std::string product;
    OrderBookType orderType;
    std::string username;
};
```



```
// =====
// ==                               Simulator.h                               ==
// =====

#pragma once

#include <string>
#include <random>
#include "OrderBook.h"
#include "UserWallet.h"
#include "TransactionLog.h"

// Task 4: THIS IS MY UNDERSTANDING OF TASK 4 i was pretty confused of the requirements i
// n task 4 and hope i understood
// it correctly

// generates 50 simulated orders (5 asks + 5 bids per product) for the
// logged-in user, prices derived from the last historical price +/- variation,
// all stamped with the current system time. Settles directly against the
// user's wallet at placement (ask credits, bid debits) and logs every order.

class Simulator {
public:
    /** Setup simulator with book, wallet and log. */
    Simulator(OrderBook& book, UserWallet& wallet, TransactionLog& txLog);

    /** Run the auto trader for this user. */
    std::string run(const std::string& username);

private:
    /** Create a fake timestamp. */
    std::string makeTimestamp();
    /** Change price a bit randomly. */
    double perturb(double base);
    /** Decide how much to buy or sell. */
    double pickAmount(double price, double balance);
    OrderBook& book;
    UserWallet& wallet;
    TransactionLog& txLog;

    std::mt19937 gen{std::random_device{}}();
};
```

```

// =====
// ==                      TransactionLog.h                      ==
// =====

#pragma once
#include <string>
#include <vector>

struct Trade { std::string timestamp, product, orderType; double price, amount; std::string username; };

class TransactionLog {
public:
    /** Setup where to save the trade history. */
    explicit TransactionLog(const std::string& filename);

    /** Save one trade and update wallet. */
    void log(const Trade& t);                                // append + adjust wallet
    /** Show last few trades for this user. */
    void printRecent(std::string username, int n = 5);
    /** Show some statistics like total volume. */
    void printStats(std::string username);
    /** Make sure the CSV file is there. */
    void ensureFilesExist();

private:
    /** Load trades from file for this user. */
    std::vector<Trade> loadFor(std::string username);
    std::string filename;
};

```

```
// =====
// ==                                UserManager.h                                ==
// =====

#pragma once
#include "UserWallet.h"
#include <vector>
#include <string>

struct UserRecord { std::string username, fullName, email, salt, passwordHash; };

class UserManager {
public:
    /** Need wallet to give bonus. */
    UserManager(UserWallet& wallet);
    /** Ask user details and make new account. */
    bool registerUser();
    /** Create account without asking (useful for tests). */
    bool registerUserProgrammatic(const std::string& fullName, const std::string& email,
const std::string& password);
    /** Get the username that just got made. */
    std::string getLatestUsername() const;
    /** Ask for email and password to log in. */
    bool login(std::string& outUsername);
    /** Change password if user forgot it. */
    bool resetPassword();
private:
    /** Make some random string for security. */
    static std::string generateSalt();
    /** Scramble password so nobody can read it. */
    static std::string hashPassword(const std::string& pw,
                                   const std::string& salt);

    /** Make a new username that nobody use yet. */
    std::string generateUniqueUsername();
    /** Turn CSV line into user record. */
    static UserRecord stringToUr(const std::string& line);
    /** Load all users from file. */
    std::vector<UserRecord> loadAll();
    /** Add one user to the CSV file. */
    void appendToFile(const UserRecord& rec);
    /** Save everything back to file. */
    void rewriteFile();
    /** Check if email or name already used. */
    bool exists(std::string& email, std::string& fullName);
    /** Print message and wait for user to type. */
    std::string readLine(const std::string& prompt);
    /** Find user by their username. */
    UserRecord* findByUsername(const std::string& username);
    /** Make sure CSV file exists. */
    void ensureFilesExist();

    const std::string users_file = "data/USERS_REGISTER.CSV";
    std::vector<UserRecord> users;
    UserWallet& wallet;
    static constexpr double SIGNUP_BONUS = 1000.0;
};
```

```

// =====
// ==                               UserWallet.h                               ==
// =====

#pragma once
#include <string>
#include <map>
#include <vector>

class UserWallet {
public:
    /** Setup wallet and load data from file. */
    explicit UserWallet(const std::string& filename);           /
    / loads USERS_WALLET.CSV

    /** Make new account for user and give some money. */
    void createAccount(std::string username, double initialBalance); // Task 2 sign-up
bonus
    /** Check if this user got account already. */
    bool hasAccount(std::string username);
    /** Check how much money user have. */
    double getBalance(std::string username);
    /** Add money to user account. */
    void deposit(std::string username, double amount);
    /** Take money from user account. Return false if not enough. */
    bool withdraw(std::string username, double amount);          // false if insuf
ficient
    /** Ask user how much they want to put in. */
    void depositFlow(std::string username);                      // interactive pr
ompt
    /** Ask user how much they want to take out. */
    void withdrawFlow(std::string username);                     // interactive pr
ompt

private:
    /** Make sure CSV file is there. */
    void ensureFilesExist();
    /** Load all balances from file. */
    void load();
    /** Save all balances to file. */
    void save();

    std::string filename;
    std::map<std::string, double> balances;
    /** Print message and get number from user. */
    static double readAmount(const std::string& prompt);
};

```

```
// =====
// ==                               Wallet.h                               ==
// =====

#pragma once

#include <string>
#include <map>
#include "OrderBookEntry.h"
#include <iostream>

class Wallet
{
    public:
        /** Just a constructor. */
        Wallet();
        /** Put some money in. */
        void insertCurrency(std::string type, double amount);
        /** Take some money out. Return false if not enough. */
        bool removeCurrency(std::string type, double amount);

        /** Check if got enough money. */
        bool containsCurrency(std::string type, double amount);
        /** Check if can pay for this order. */
        bool canFulfillOrder(OrderBookEntry order);
        /** Update wallet after trade happen. */
        void processSale(OrderBookEntry& sale);

        /** Show wallet contents as string. */
        std::string toString();
        /** Allow printing wallet directly. */
        friend std::ostream& operator<<(std::ostream& os, Wallet& wallet);

    private:
        std::map<std::string,double> currencies;
};
```

```

// =====
// ==                      CSVReader.cpp                      ==
// =====

#include "CSVReader.h"
#include <iostream>
#include <fstream>

CSVReader::CSVReader()
{

}

std::vector<OrderBookEntry> CSVReader::readCSV(std::string csvFilename)
{
    std::vector<OrderBookEntry> entries;

    std::ifstream csvFile{csvFilename};
    std::string line;
    if (csvFile.is_open())
    {
        while(std::getline(csvFile, line))
        {
            try {
                OrderBookEntry obe = stringsToOBE(tokenise(line, ','));
                entries.push_back(obe);
            } catch(const std::exception& e)
            {
                std::cout << "CSVReader::readCSV bad data" << std::endl;
            }
        } // end of while
    }
    else {
        std::cout << "CSVReader::readCSV bad file: " << csvFilename << std::endl;
    }

    std::cout << "CSVReader::readCSV read " << entries.size() << " entries" << std::endl;
;
    return entries;
}

std::vector<std::string> CSVReader::tokenise(std::string csvLine, char separator)
{
    std::vector<std::string> tokens;
    signed int start, end;
    std::string token;
    start = csvLine.find_first_not_of(separator, 0);
    do{
        end = csvLine.find_first_of(separator, start);
        if (start == csvLine.length() || start == end) break;
        if (end >= 0) token = csvLine.substr(start, end - start);
        else token = csvLine.substr(start, csvLine.length() - start);
        tokens.push_back(token);
        start = end + 1;
    }while(end > 0);

    return tokens;
}

OrderBookEntry CSVReader::stringsToOBE(std::vector<std::string> tokens)
{
    double price, amount;

    if (tokens.size() != 5) // bad
    {
        std::cout << "Bad line " << std::endl;
        throw std::exception{};
    }
    // we have 5 tokens
    try {

```



```
// =====  
// ==                      Candlestick.cpp                      ==  
// =====
```

```
#include "Candlestick.h"
```

```
Candlestick::Candlestick(std::string timestamp,  
                          std::string product,  
                          OrderBookType orderType,  
                          double open,  
                          double high,  
                          double low,  
                          double close)
```

```
: timestamp(timestamp),  
  product(product),  
  orderType(orderType),  
  open(open),  
  high(high),  
  low(low),  
  close(close)  
{}
```

```
double Candlestick::getClose() const {  
    return close;  
}
```

```
double Candlestick::getOpen() const {  
    return open;  
}
```

```
double Candlestick::getHigh() const {  
    return high;  
}
```

```
double Candlestick::getLow() const {  
    return low;  
}
```

```
std::string Candlestick::getProduct() const {  
    return product;  
}
```

```
std::string Candlestick::getTime() const {  
    return timestamp;  
}
```

```
OrderBookType Candlestick::getOrderType() const {  
    return orderType;  
}
```



```

// =====
// ==                  CandlestickCalculator.cpp                  ==
// =====

#include "CandlestickCalculator.h"
#include "Candlestick.h"
#include <map>
#include <array>

std::vector<Candlestick> CandlestickCalculator::calculate(const std::vector<OrderBookEntry>& orders,
    std::string product,
    OrderBookType orderType,
    TimeGroup timeGroup,
    bool includeBoth) {

    /**
    [0] = open
    [1] = high
    [2] = low
    [3] = close
    */
    std::map<std::string, std::array<double, 4>> candleMap;

    for (const OrderBookEntry& order : orders) {
        bool matchType = includeBoth ? (order.orderType == OrderBookType::ask || order.orderType == OrderBookType::bid)
                                     : (order.orderType == orderType);
        if (order.product == product && matchType) {
            std::string period;
            switch (timeGroup) {
                case TimeGroup::Daily:
                    period = order.timestamp.substr(0, 10);
                    break;
                case TimeGroup::Monthly:
                    period = order.timestamp.substr(0, 7);
                    break;
                case TimeGroup::Yearly:
                    period = order.timestamp.substr(0, 4);
                    break;
            }
            if (candleMap.find(period) == candleMap.end()) {
                candleMap[period] = std::array<double, 4>{order.price, order.price, order.price, order.price};
            }
            else {
                if (order.price > candleMap[period][1]) {
                    candleMap[period][1] = order.price;
                } else if (order.price < candleMap[period][2]) {
                    candleMap[period][2] = order.price;
                }
                candleMap[period][3] = order.price;
            }
        }
    }

    std::vector<Candlestick> candles;
    candles.reserve(candleMap.size());

    for (const auto& candleData: candleMap) {
        candles.push_back(Candlestick{
            candleData.first,
            product,
            orderType,
            candleData.second[0],
            candleData.second[1],
            candleData.second[2],
            candleData.second[3]});
    }
}

```

```
    return candles;  
}
```

```
// =====
// ==                      Candlestickplotter.cpp                      ==
// =====

#include "Candlestickplotter.h"
#include <iostream>
#include <sstream>
#include <iomanip>
#include <cmath>
#include <algorithm>

/**
 * THIS COMMENT WAS AI GENERATED FOR ME TO HAVE A CLEARLY STATED IDEA ON WHAT I WANTED TO
 * DO
 * Grid approach: we build a 2D char grid (rows x columns) using single-byte
 * MARKER characters, then translate markers to coloured glyphs at print time.
 * (We can't store multi-byte glyphs like â\226\210 directly in the char grid without
 * breaking column alignment.)
 *
 * Markers:  'U' = up-candle body    -> green â\226\210
 *           'D' = down-candle body -> red   â\226\210
 *           'W' = wick              -> â\224\202 (coloured like its candle)
 *           ' ' = empty
 *
 * ANSI colours work in the CLion console and most modern terminals.
 * If output ever looks garbled (old Windows cmd.exe), set USE_COLOUR = false.
 */

namespace {
    const int COL_WIDTH = 6;          // columns per candle (3-wide body + gap)
    const bool USE_COLOUR = true;

    const std::string GREEN = "\033[32m";
    const std::string RED   = "\033[31m";
    const std::string RESET = "\033[0m";
}

/**
 * Work out which row the price belongs to.
 */
int CandlestickPlotter::priceToRow(double p, double lo, double hi, int height)
{
    double norm = (p - lo) / (hi - lo);          // 0.0 .. 1.0
    int row = static_cast<int>(std::lround(norm * (height - 1)));
    if (row < 0) row = 0;                        // clamp against rounding drift
    if (row > height - 1) row = height - 1;
    return row;
}

/**
 * Make date string shorter so it fit.
 */
std::string CandlestickPlotter::dateLabel(const std::string& time)
{
    if (time.length() >= 10) return time.substr(5, 5); // daily: MM-DD
    if (time.length() == 7)  return time.substr(2);    // monthly: YY-MM
    return time;                                       // yearly: YYYY
}

/**
 * Format price properly so easier to read.
 */
std::string CandlestickPlotter::formatPrice(double v)
{
    std::ostringstream ss;
    if (v >= 1000.0)    ss << std::fixed << std::setprecision(1) << v;
    else if (v >= 1.0)  ss << std::fixed << std::setprecision(2) << v;
    else if (v >= 0.001) ss << std::fixed << std::setprecision(5) << v;
    else                ss << std::scientific << std::setprecision(2) << v;
    return ss.str();
}

```

```

/**
 * Print the graph to terminal. Use colours if can.
 */
void CandlestickPlotter::plot(const std::vector<Candlestick>& candles,
                             int height, int maxCandles)
{
    if (candles.empty()) {
        std::cout << "No candlestick data to plot." << std::endl;
        return;
    }

    //keep only the most recent maxCandles
    std::vector<Candlestick> shown = candles;
    if (static_cast<int>(shown.size()) > maxCandles) {
        shown.assign(candles.end() - maxCandles, candles.end());
        std::cout << "(showing the most recent " << maxCandles
                  << " of " << candles.size() << " candles)" << std::endl;
    }
    const int n = static_cast<int>(shown.size());

    //price range across all shown candles
    double lo = shown[0].getLow();
    double hi = shown[0].getHigh();
    for (const Candlestick& c : shown) {
        lo = std::min(lo, c.getLow());
        hi = std::max(hi, c.getHigh());
    }
    if (hi == lo) hi = lo + 1.0;

    // build marker grid: row 0 = botto
    const int width = n * COL_WIDTH;
    std::vector<std::string> grid(height, std::string(width, ' '));

    for (int i = 0; i < n; ++i) {
        const Candlestick& c = shown[i];
        const int centre = i * COL_WIDTH + COL_WIDTH / 2;
        const bool up = c.getClose() >= c.getOpen();
        const char body = up ? 'U' : 'D';

        const int rHigh = priceToRow(c.getHigh(), lo, hi, height);
        const int rLow = priceToRow(c.getLow(), lo, hi, height);
        const int rTop = priceToRow(std::max(c.getOpen(), c.getClose()), lo, hi, height)
;
        const int rBot = priceToRow(std::min(c.getOpen(), c.getClose()), lo, hi, height)
;

        for (int r = rLow; r <= rHigh; ++r) // wick first...
            grid[r][centre] = 'W';
        for (int r = rBot; r <= rTop; ++r) // ...body drawn over it
            for (int col = centre - 1; col <= centre + 1; ++col)
                grid[r][col] = body;
    }

    //print top-down with a y-axis label every 4th row
    const int Y_WIDTH = 12;
    for (int r = height - 1; r >= 0; --r) {
        if (r % 4 == 0) {
            double price = lo + (hi - lo) * r / (height - 1);
            std::cout << std::right << std::setw(Y_WIDTH) << formatPrice(price) << " â
\224 "
;
        } else {
            std::cout << std::string(Y_WIDTH, ' ') << " â\224\202 ";
        }
    }

    for (int col = 0; col < width; ++col) {
        const char m = grid[r][col];
        // colour follows the candle this column belongs to
        const int candleIdx = col / COL_WIDTH;
        const bool up = shown[candleIdx].getClose() >= shown[candleIdx].getOpen();
        const std::string& colour = up ? GREEN : RED;
    }
}

```

```

        if (m == 'U')          std::cout << (USE_COLOUR ? colour + "\226\210" + RESET :
"\226\210");
        else if (m == 'D') std::cout << (USE_COLOUR ? colour + "\226\210" + RESET :
"\226\210");
        else if (m == 'W') std::cout << (USE_COLOUR ? colour + "\224\202" + RESET :
"\224\202");
        else
            std::cout << ' ';
    }
    std::cout << "\n";
}

//x-axis and date labels centred under each candle
std::cout << std::string(Y_WIDTH, ' ') << " \224\224" << std::string(width, '-') <<
"\n";
std::cout << std::string(Y_WIDTH, ' ') << "  ";
for (int i = 0; i < n; ++i) {
    std::string label = dateLabel(shown[i].getTime());
    if (static_cast<int>(label.size()) > COL_WIDTH)
        label = label.substr(0, COL_WIDTH);
    const int pad = (COL_WIDTH - static_cast<int>(label.size())) / 2;
    std::cout << std::string(pad, ' ') << label
               << std::string(COL_WIDTH - pad - label.size(), ' ');
}
std::cout << std::endl;
}

```

```
// =====
// ==                                MerkelMain.cpp                                ==
// =====

#include "MerkelMain.h"
#include <iostream>
#include <iomanip>
#include "CSVReader.h"

MerkelMain::MerkelMain()
{
}

void MerkelMain::init()
{
    currentTime = orderBook.getEarliestTime();

    bool running = true;
    while (running)
    {
        if (currentUser.empty())                // ---- logged out ----
        {
            printLoginMenu();
            int option = getUserOption(5);
            if (option == -1) break;              // EOF / stream closed
            processLoginOption(option, running);
        }
        else                                    // ---- logged in ----
        {
            printMainMenu();
            int option = getUserOption(10);
            if (option == -1) break;
            processMainOption(option);
        }
    }
    std::cout << "Goodbye." << std::endl;
}

// ===== menus =====

void MerkelMain::printLoginMenu()
{
    std::cout << "===== Welcome to MerkelRex =====" << std::endl;
    std::cout << "1: Register a new account" << std::endl;
    std::cout << "2: Log in" << std::endl;
    std::cout << "3: Reset password" << std::endl;
    std::cout << "4: Help" << std::endl;
    std::cout << "5: Exit" << std::endl;
    std::cout << "===== " << std::endl;
}

void MerkelMain::printMainMenu()
{
    std::cout << "===== MerkelRex =====" << std::endl;
    std::cout << "1: Print candlestick (OHLC) data" << std::endl;
    std::cout << "2: Plot candlestick chart" << std::endl;
    std::cout << "3: Deposit funds" << std::endl;
    std::cout << "4: Withdraw funds" << std::endl;
    std::cout << "5: Recent transactions (last 5)" << std::endl;
    std::cout << "6: Activity statistics" << std::endl;
    std::cout << "7: Simulate trading (50 orders)" << std::endl;
    std::cout << "8: Go to next timeframe (match)" << std::endl;
    std::cout << "9: Help" << std::endl;
    std::cout << "10: Log out" << std::endl;
    std::cout << "===== " << std::endl;
    std::cout << "Logged in as: " << currentUser
        << " | Balance: " << std::fixed << std::setprecision(2)
        << userWallet.getBalance(currentUser) << std::endl;
    std::cout << "Current time: " << currentTime << std::endl;
}

```

```

// Validated menu input (Task 5): getline + stoi in try/catch + range check,
// loops until a valid option is entered - never crashes on bad input.
int MerkelMain::getUserOption(int maxOption)
{
    while (true)
    {
        std::cout << "Type an option (1-" << maxOption << "): ";
        std::string line;
        if (!std::getline(std::cin, line)) return -1;    // stream closed
        try
        {
            int option = std::stoi(line);
            if (option >= 1 && option <= maxOption) return option;
        }
        catch (const std::exception& e) { /* fall through to retry */ }
        std::cout << "Invalid choice, please try again." << std::endl;
    }
}

void MerkelMain::processLoginOption(int option, bool& running)
{
    if (option == 1) doRegister();
    else if (option == 2) doLogin();
    else if (option == 3) doResetPassword();
    else if (option == 4) printLoginHelp();
    else if (option == 5) running = false;
}

void MerkelMain::processMainOption(int option)
{
    if (option == 1) printCandlestickData();
    else if (option == 2) doPlotChart();
    else if (option == 3) userWallet.depositFlow(currentUser);
    else if (option == 4) userWallet.withdrawFlow(currentUser);
    else if (option == 5) txLog.printRecent(currentUser);
    else if (option == 6) txLog.printStats(currentUser);
    else if (option == 7) doSimulation();
    else if (option == 8) gotoNextTimeframe();
    else if (option == 9) printMainHelp();
    else if (option == 10) { std::cout << "Logging out " << currentUser << std::endl; currentUser = ""; }
}

// ===== help =====

void MerkelMain::printLoginHelp()
{
    std::cout << "\n--- Help ---" << std::endl;
    std::cout << "Register: create an account; you'll be given a 10-digit username." << std::endl;
    std::cout << "Log in:   enter that username and your password to start trading." << std::endl;
    std::cout << "Reset:    change your password by confirming your username + email." << std::endl;
    std::cout << "Exit:     close the program.\n" << std::endl;
}

void MerkelMain::printMainHelp()
{
    std::cout << "\n--- Help ---" << std::endl;
    std::cout << "1/2: view market history as an OHLC table or a candlestick chart." << std::endl;
    std::cout << "3/4: deposit or withdraw funds from your wallet." << std::endl;
    std::cout << "5/6: review your recent trades and activity statistics." << std::endl;
    std::cout << "7:    generate 50 simulated orders priced from the historical data." << std::endl;
    std::cout << "8:    advance the clock and match orders in the current timeframe.\n" << std::endl;
}

// ===== Task 2 handlers =====

```

```

void MerkelMain::doRegister() { userManager.registerUser(); }

void MerkelMain::doLogin()
{
    std::string user;
    if (userManager.login(user)) currentUser = user;    // session starts
}

void MerkelMain::doResetPassword() { userManager.resetPassword(); }

// ===== Task 1 =====

// Shared prompt used by both the OHLC table and the chart, so the
// product/order-type/time-group questions live in one place.
std::vector<Candlestick> MerkelMain::getCandlesFromUser(bool askType)
{
    std::string product;
    std::cout << "Enter product (e.g. ETH/BTC): ";
    std::getline(std::cin, product);

    OrderBookType orderType = OrderBookType::unknown;
    if (askType) {
        std::string orderTypeInput;
        std::cout << "Enter order type (ask/bid): ";
        std::getline(std::cin, orderTypeInput);
        orderType = OrderBookEntry::stringToOrderBookType(orderTypeInput);
    }

    std::string timeGroupInput;
    std::cout << "Enter time group (1 = Daily, 2 = Monthly, 3 = Yearly): ";
    std::getline(std::cin, timeGroupInput);
    TimeGroup timeGroup = TimeGroup::Daily;
    try {
        switch (std::stoi(timeGroupInput)) {
            case 2: timeGroup = TimeGroup::Monthly; break;
            case 3: timeGroup = TimeGroup::Yearly; break;
            default: timeGroup = TimeGroup::Daily; break;
        }
    } catch (const std::exception& e) { timeGroup = TimeGroup::Daily; }

    return CandlestickCalculator::calculate(
        orderBook.getOrderBook(), product, orderType, timeGroup, !askType);
}

void MerkelMain::printCandlestickData()
{
    std::vector<Candlestick> candles = getCandlesFromUser();

    std::string startDate, endDate;
    std::cout << "Enter start date (e.g. 2020-03-17, leave blank to skip): ";
    std::getline(std::cin, startDate);
    std::cout << "Enter end date (e.g. 2020-03-17, leave blank to skip): ";
    std::getline(std::cin, endDate);

    std::cout << std::left
        << std::setw(20) << "Date" << std::setw(15) << "Open"
        << std::setw(15) << "High" << std::setw(15) << "Low"
        << std::setw(15) << "Close" << std::endl;
    std::cout << std::string(80, '-') << std::endl;

    for (const Candlestick& candle : candles) {
        if (!startDate.empty() && candle.getTime() < startDate) continue;
        if (!endDate.empty() && candle.getTime() > endDate) continue;
        std::cout << std::left
            << std::setw(20) << candle.getTime()
            << std::setw(15) << candle.getOpen()
            << std::setw(15) << candle.getHigh()
            << std::setw(15) << candle.getLow()
            << std::setw(15) << candle.getClose()
            << std::endl;
    }
}

```



```

    }
}

void MerkelMain::doPlotChart()
{
    std::vector<Candlestick> candles = getCandlesFromUser(false);
    CandlestickPlotter::plot(candles);
}

// ===== trading handlers =====

void MerkelMain::doSimulation()
{
    std::string timestamp = simulator.run(currentUser);
    currentTime = timestamp;    // advance so 'next timeframe' lands on new orders
}

void MerkelMain::gotoNextTimeframe()
{
    std::cout << "Going to next time frame... " << std::endl;
    for (const std::string& p : orderBook.getKnownProducts())
    {
        std::cout << "matching " << p << std::endl;
        std::vector<OrderBookEntry> entries = orderBook.matchAsksToBids(p, currentTime);
        std::cout << "got " << entries.size() << " matches" << std::endl;
        for (OrderBookEntry& e : entries)
        {
            std::cout << "Match: " << e.price << " amount " << e.amount << std::endl;
            if (e.username == currentUser)
            {
                std::string type = (e.orderType == OrderBookType::asksale) ? "ask" : "bid";
                txLog.log({ e.timestamp, e.product, type, e.price, e.amount, e.username });
            }
        }
    }
    currentTime = orderBook.getNextTime(currentTime);
}

```

```

// =====
// ==                                OrderBook.cpp                                ==
// =====

#include "OrderBook.h"
#include "CSVReader.h"
#include <map>
#include <algorithm>
#include <iostream>

/** construct, reading a csv data file */
OrderBook::OrderBook(std::string filename)
{
    orders = CSVReader::readCSV(filename);
}

/** return vector of all know products in the dataset*/
std::vector<std::string> OrderBook::getKnownProducts()
{
    std::vector<std::string> products;

    std::map<std::string, bool> prodMap;

    for (OrderBookEntry& e : orders)
    {
        prodMap[e.product] = true;
    }

    // now flatten the map to a vector of strings
    for (auto const& e : prodMap)
    {
        products.push_back(e.first);
    }

    return products;
}

/** return vector of Orders according to the sent filters*/
std::vector<OrderBookEntry> OrderBook::getOrders(OrderBookType type,
                                                std::string product,
                                                std::string timestamp)
{
    std::vector<OrderBookEntry> orders_sub;
    for (OrderBookEntry& e : orders)
    {
        if (e.orderType == type &&
            e.product == product &&
            e.timestamp == timestamp )
        {
            orders_sub.push_back(e);
        }
    }
    return orders_sub;
}

double OrderBook::getHighPrice(std::vector<OrderBookEntry>& orders)
{
    double max = orders[0].price;
    for (OrderBookEntry& e : orders)
    {
        if (e.price > max) max = e.price;
    }
    return max;
}

double OrderBook::getLowPrice(std::vector<OrderBookEntry>& orders)
{
    double min = orders[0].price;
    for (OrderBookEntry& e : orders)

```

```

    {
        if (e.price < min) min = e.price;
    }
    return min;
}

std::string OrderBook::getEarliestTime()
{
    return orders[0].timestamp;
}

std::string OrderBook::getNextTime(std::string timestamp)
{
    std::string next_timestamp = "";
    for (OrderBookEntry& e : orders)
    {
        if (e.timestamp > timestamp)
        {
            next_timestamp = e.timestamp;
            break;
        }
    }
    if (next_timestamp == "")
    {
        next_timestamp = orders[0].timestamp;
    }
    return next_timestamp;
}

void OrderBook::insertOrder(OrderBookEntry& order)
{
    orders.push_back(order);
    std::sort(orders.begin(), orders.end(), OrderBookEntry::compareByTimestamp);
}

std::vector<OrderBookEntry> OrderBook::matchAsksToBids(std::string product, std::string t
imestamp)
{
    // asks = orderbook.asks
    std::vector<OrderBookEntry> asks = getOrders(OrderBookType::ask,
                                                    product,
                                                    timestamp);

    // bids = orderbook.bids
    std::vector<OrderBookEntry> bids = getOrders(OrderBookType::bid,
                                                    product,
                                                    timestamp);

    // sales = []
    std::vector<OrderBookEntry> sales;

    // I put in a little check to ensure we have bids and asks
    // to process.
    if (asks.size() == 0 || bids.size() == 0)
    {
        std::cout << " OrderBook::matchAsksToBids no bids or asks" << std::endl;
        return sales;
    }

    // sort asks lowest first
    std::sort(asks.begin(), asks.end(), OrderBookEntry::compareByPriceAsc);
    // sort bids highest first
    std::sort(bids.begin(), bids.end(), OrderBookEntry::compareByPriceDesc);
    // for ask in asks:
    std::cout << "max ask " << asks[asks.size()-1].price << std::endl;
    std::cout << "min ask " << asks[0].price << std::endl;
    std::cout << "max bid " << bids[0].price << std::endl;
    std::cout << "min bid " << bids[bids.size()-1].price << std::endl;

    for (OrderBookEntry& ask : asks)
    {
        //      for bid in bids:

```

```

for (OrderBookEntry& bid : bids)
{
//      if bid.price >= ask.price # we have a match
    if (bid.price >= ask.price)
    {
//          sale = new order()
//          sale.price = ask.price
        OrderBookEntry sale{ask.price, 0, timestamp,
            product,
            OrderBookType::asksale};

        if (bid.username == "simuser")
        {
            sale.username = "simuser";
            sale.orderType = OrderBookType::bidsale;
        }
        if (ask.username == "simuser")
        {
            sale.username = "simuser";
            sale.orderType = OrderBookType::asksale;
        }

//          # now work out how much was sold and
//          # create new bids and asks covering
//          # anything that was not sold
//          if bid.amount == ask.amount: # bid completely clears ask
        if (bid.amount == ask.amount)
        {
//              sale.amount = ask.amount
            sale.amount = ask.amount;
//              sales.append(sale)
            sales.push_back(sale);
//              bid.amount = 0 # make sure the bid is not processed again
            bid.amount = 0;
//              # can do no more with this ask
//              # go onto the next ask
//              break
            break;
        }
//          if bid.amount > ask.amount: # ask is completely gone slice the bid
        if (bid.amount > ask.amount)
        {
//              sale.amount = ask.amount
            sale.amount = ask.amount;
//              sales.append(sale)
            sales.push_back(sale);
//              # we adjust the bid in place
//              # so it can be used to process the next ask
//              bid.amount = bid.amount - ask.amount
            bid.amount = bid.amount - ask.amount;
//              # ask is completely gone, so go to next ask
//              break
            break;
        }

//          if bid.amount < ask.amount # bid is completely gone, slice the ask
        if (bid.amount < ask.amount &&
            bid.amount > 0)
        {
//              sale.amount = bid.amount
            sale.amount = bid.amount;
//              sales.append(sale)
            sales.push_back(sale);
//              # update the ask
//              # and allow further bids to process the remaining amount
//              ask.amount = ask.amount - bid.amount
            ask.amount = ask.amount - bid.amount;
//              bid.amount = 0 # make sure the bid is not processed again
            bid.amount = 0;
//              # some ask remains so go to the next bid

```

```
        //                continue  
        continue;  
    }  
}  
}  
return sales;  
}  
  
const std::vector<OrderBookEntry>& OrderBook::getOrderBook() const {  
    return orders;  
}
```

```
// =====
// ==                                OrderBookEntry.cpp                                ==
// =====

#include "OrderBookEntry.h"

OrderBookEntry::OrderBookEntry( double _price,
                                double _amount,
                                std::string _timestamp,
                                std::string _product,
                                OrderBookType _orderType,
                                std::string _username)
: price(_price),
  amount(_amount),
  timestamp(_timestamp),
  product(_product),
  orderType(_orderType),
  username(_username)
{

}

OrderBookType OrderBookEntry::stringToOrderBookType(std::string s)
{
    if (s == "ask")
    {
        return OrderBookType::ask;
    }
    if (s == "bid")
    {
        return OrderBookType::bid;
    }
    return OrderBookType::unknown;
}

```

```
// =====
// ==                               Simulator.cpp                               ==
// =====

#include "Simulator.h"
#include "OrderBookEntry.h"
#include <map>
#include <ctime>
#include <iomanip>
#include <sstream>
#include <iostream>

Simulator::Simulator(OrderBook& book, UserWallet& wallet, TransactionLog& txLog)
    : book(book), wallet(wallet), txLog(txLog) {}

std::string Simulator::makeTimestamp() {
    std::time_t t = std::time(nullptr);
    std::ostringstream ss;
    ss << std::put_time(std::localtime(&t), "%Y/%m/%d %H:%M:%S");
    return ss.str();
}

double Simulator::perturb(double base) {
    std::uniform_real_distribution<double> dist(0.95, 1.10);
    return base * dist(gen);
}

double Simulator::pickAmount(double price, double balance) {
    std::uniform_real_distribution<double> dist(0.001, 0.01);
    return (balance * dist(gen)) / price;
}

std::string Simulator::run(const std::string& username) {
    std::string timestamp = makeTimestamp();

    std::map<std::string, double> lastAsk, lastBid;
    for (const OrderBookEntry& e : book.getOrderBook()) {
        if (e.username != "dataset") continue;
        if (e.orderType == OrderBookType::ask) lastAsk[e.product] = e.price;
        if (e.orderType == OrderBookType::bid) lastBid[e.product] = e.price;
    }

    int made = 0;
    for (const std::string& product : book.getKnownProducts()) {
        double baseAsk = lastAsk.count(product) ? lastAsk[product] : 0.0;
        double baseBid = lastBid.count(product) ? lastBid[product] : 0.0;
        if (baseAsk <= 0.0 || baseBid <= 0.0) {
            std::cout << "Skipping " << product << " - no baseline price." << std::endl;
            continue;
        }

        for (int i = 0; i < 5; ++i) {

            double askPrice = perturb(baseAsk);
            double askAmt = pickAmount(askPrice, wallet.getBalance(username));
            OrderBookEntry askEntry{askPrice, askAmt, timestamp, product,
                                    OrderBookType::ask, username};
            book.insertOrder(askEntry);
            txLog.log(Trade{timestamp, product, "ask", askPrice, askAmt, username});
            wallet.deposit(username, askPrice * askAmt);
            ++made;

            double bidPrice = perturb(baseBid);
            double bidAmt = pickAmount(bidPrice, wallet.getBalance(username));
            double cost = bidPrice * bidAmt;
            if (!wallet.withdraw(username, cost)) {
                std::cout << "Insufficient balance - skipping bid for " << product << std::endl;
            }
        }
    }
}

```

```

::endl;
        continue;
    }
    OrderBookEntry bidEntry{bidPrice, bidAmt, timestamp, product,
        OrderBookType::bid, username};
    book.insertOrder(bidEntry);
    txLog.log(Trade{timestamp, product, "bid", bidPrice, bidAmt, username});
    ++made;
}

std::cout << "Simulation complete: " << made
    << " orders placed at " << timestamp << std::endl;
return timestamp;
}

```



```

// =====
// ==                      TransactionLog.cpp                      ==
// =====

#include "../TransactionLog.h"
#include "CSVReader.h"
#include <fstream>
#include <iostream>
#include <iomanip>
#include <map>
#include <set>

TransactionLog::TransactionLog(const std::string& filename) : filename(filename) {
    ensureFilesExist();
}

void TransactionLog::ensureFilesExist() {
    std::ifstream check{filename};
    if (!check.is_open()) {
        std::ofstream create{filename};
    }
}

void TransactionLog::log(const Trade& t) {
    std::ofstream out{filename, std::ios::app};           // append mode
    if (!out.is_open()) {
        std::cerr << "TransactionLog::log bad file: " << filename << std::endl;
        return;
    }
    out << t.timestamp << "," << t.product << "," << t.orderType << ","
        << std::fixed << std::setprecision(2) << t.price << ","
        << t.amount << "," << t.username << "\n";
}

std::vector<Trade> TransactionLog::loadFor(std::string username) {
    std::vector<Trade> trades;
    std::ifstream file{filename};
    if (!file.is_open()) return trades;
    std::string line;
    while (std::getline(file, line)) {
        if (line.empty()) continue;
        std::vector<std::string> tk = CSVReader::tokenise(line, ',');
        if (tk.size() != 6) continue;
        try {
            Trade t{ tk[0], tk[1], tk[2], std::stod(tk[3]), std::stod(tk[4]), tk[5] };
            if (t.username == username) trades.push_back(t);
        } catch (const std::exception& e) { }
    }
    return trades;
}

void TransactionLog::printRecent(std::string username, int n) {
    std::vector<Trade> trades = loadFor(username);
    std::cout << "--- Last " << n << " transactions ---" << std::endl;
    if (trades.empty()) { std::cout << "No transactions yet." << std::endl; return; }

    int start = static_cast<int>(trades.size()) - n;
    if (start < 0) start = 0;
    std::cout << std::left
        << std::setw(26) << "Timestamp" << std::setw(10) << "Product"
        << std::setw(6) << "Type" << std::setw(12) << "Price"
        << std::setw(12) << "Amount" << std::endl;
    for (int i = start; i < static_cast<int>(trades.size()); ++i) {
        const Trade& t = trades[i];
        std::cout << std::left
            << std::setw(26) << t.timestamp << std::setw(10) << t.product
            << std::setw(6) << t.orderType << std::setw(12) << t.price
            << std::setw(12) << t.amount << std::endl;
    }
}

```

```

void TransactionLog::printStats(std::string username) {
    std::vector<Trade> trades = loadFor(username);
    if (trades.empty()) { std::cout << "No transactions yet." << std::endl; return; }

    std::cout << "From (YYYY/MM/DD, blank = all): ";
    std::string from; std::getline(std::cin, from);
    std::cout << "To (YYYY/MM/DD, blank = all): ";
    std::string to; std::getline(std::cin, to);

    int totalAsks = 0, totalBids = 0;
    std::map<std::string, int> asksByProduct;
    std::map<std::string, int> bidsByProduct;
    double moneySpent = 0.0;

    for (const Trade& t : trades) {
        if (!from.empty() && t.timestamp < from) continue;
        if (!to.empty() && t.timestamp > to) continue;

        if (t.orderType == "ask") {
            ++totalAsks;
            ++asksByProduct[t.product];
        } else if (t.orderType == "bid") {
            ++totalBids;
            ++bidsByProduct[t.product];
            moneySpent += t.price * t.amount;
        }
    }

    std::cout << "\n--- Activity summary for " << username << " ---" << std::endl;
    if (!from.empty() || !to.empty()) {
        std::cout << "Timeframe: " << (from.empty() ? "start" : from)
            << " to " << (to.empty() ? "now" : to) << std::endl;
    }
    std::cout << "Total asks: " << totalAsks << "    Total bids: " << totalBids << std::en
dl;

    std::cout << "\nPer product:" << std::endl;
    std::set<std::string> products;
    for (const auto& p : asksByProduct) products.insert(p.first);
    for (const auto& p : bidsByProduct) products.insert(p.first);
    for (const std::string& prod : products) {
        std::cout << " " << std::left << std::setw(10) << prod
            << asksByProduct[prod] << " asks, "
            << bidsByProduct[prod] << " bids" << std::endl;
    }

    std::cout << "\nTotal money spent (bids) in timeframe: "
        << std::fixed << std::setprecision(2) << moneySpent << std::endl;
}

```

```
// =====
// ==                                UserManager.cpp                                ==
// =====

#include "UserManager.h"
#include "UserWallet.h"
#include <iostream>
#include <functional>
#include <random>
#include <iomanip>
#include <sstream>
#include <string>
#include <limits>
#include <fstream>

#include "CSVReader.h"

UserManager::UserManager(UserWallet& wallet) : wallet(wallet) {
    ensureFilesExist();
    loadAll();
};

bool UserManager::registerUser() {
    std::cout << "--- Register ---" << std::endl;
    std::string fullName = readLine("Full name: ");
    std::string email = readLine("Email: ");
    std::string password = readLine("Password: ");

    return registerUserProgrammatic(fullName, email, password);
}

bool UserManager::registerUserProgrammatic(const std::string& fullName,
                                           const std::string& email,
                                           const std::string& password) {
    if (fullName.empty() || email.empty() || password.empty()) {
        std::cout << "All fields are required." << std::endl;
        return false;
    }
    if (email.find('@') == std::string::npos) {
        std::cout << "That does not look like a valid email." << std::endl;
        return false;
    }

    if (fullName.find(',') != std::string::npos ||
        email.find(',') != std::string::npos ||
        password.find(',') != std::string::npos) {
        std::cout << "Commas are not allowed in any field." << std::endl;
        return false;
    }

    if (exists(const_cast<std::string&>(email), const_cast<std::string&>(fullName))) {
        std::cout << "An account with this name and email already exists." << std::endl;
        return false;
    }

    std::string username = generateUniqueUsername();
    std::string salt = generateSalt();
    std::string hash = hashPassword(password, salt);

    UserRecord rec{username, fullName, email, salt, hash};
    users.push_back(rec);
    appendToFile(rec);

    wallet.createAccount(username, SIGNUP_BONUS);

    std::cout << ">>> Registered! Your username is: " << username
              << " - keep it safe to log in. <<<" << std::endl;
    return true;
}
```

```

}

std::string UserManager::getLatestUsername() const {
    if (users.empty()) return "";
    return users.back().username;
}

bool UserManager::login(std::string& outUsername) {
    std::cout << "--- Login ---" << std::endl;
    for (int attempt = 1; attempt <= 3; ++attempt) {
        std::string username = readLine("Username: ");
        std::string password = readLine("Password: ");

        UserRecord* rec = findByUsername(username);
        if (rec != nullptr && hashPassword(password, rec->salt) == rec->passwordHash) {
            outUsername = username;
            std::cout << "Welcome back, " << rec->fullName << "!" << std::endl;
            return true;
        }
        std::cout << "Invalid credentials (attempt " << attempt << " of 3)." << std::endl;
    }
    return false;
}

bool UserManager::resetPassword() {
    std::cout << "--- Reset password ---" << std::endl;
    std::cout << "Confirm your identity." << std::endl;
    std::string username = readLine("Username: ");
    std::string email = readLine("Email: ");

    UserRecord* rec = findByUsername(username);
    if (rec == nullptr || rec->email != email) {
        std::cout << "Details do not match our records." << std::endl;
        return false;
    }

    std::string newPass = readLine("New password: ");
    std::string confirm = readLine("Confirm new password: ");
    if (newPass.empty()) {
        std::cout << "Password cannot be empty." << std::endl;
        return false;
    }
    if (newPass != confirm) {
        std::cout << "Passwords do not match." << std::endl;
        return false;
    }

    rec->salt = generateSalt();
    rec->passwordHash = hashPassword(newPass, rec->salt);
    rewriteFile();
    return true;
}

std::string UserManager::generateSalt() {
    std::random_device rd;
    std::mt19937_64 gen(rd());

    std::uniform_int_distribution<uint64_t> dist(0, std::numeric_limits<uint64_t>::max());

    const uint64_t salt_value = dist(gen);

    std::stringstream ss;
    ss << std::setfill('0') << std::setw(16) << std::hex << salt_value;

    return ss.str();
};

```

```

std::string UserManager::hashPassword(const std::string& pw, const std::string& salt) {
    return std::to_string( std::hash<std::string>{}(salt + pw) );
};

std::string UserManager::generateUniqueUsername() {
    thread_local std::random_device rd;
    thread_local std::mt19937_64 gen(rd());
    std::uniform_int_distribution<long long> distr(1000000000LL, 9999999999LL);

    while (true) {
        std::string candidate = std::to_string(distr(gen));

        bool taken = false;
        for (const UserRecord& r : users) { // this->users
            if (r.username == candidate) { taken = true; break; }
        }
        if (!taken) return candidate;
    }
}

std::vector<UserRecord> UserManager::loadAll() {

    std::ifstream csvFile{this->users_file};
    this->users.clear();

    if (csvFile.is_open())
    {
        std::string line;
        while(std::getline(csvFile, line))
        {
            try {

                UserRecord ur = stringToUr(line);
                this->users.push_back(ur);
            }catch(const std::exception& e)
            {
                std::cout << "UserManager::loadAll bad data" << std::endl;
            }
        }
    }
    else {
        std::cerr << "UserManager::loadAll bad file: " << this->users_file << std::endl;
    }

    std::cout << "UserManager::loadAll read " << this->users.size() << "entries" << std::endl;
    return this->users;
};

UserRecord UserManager::stringToUr(const std::string& line) {
    std::vector<std::string> const tokens = CSVReader::tokenise(line, ',');
    if (tokens.size() != 5)
    {
        std::cout << "Bad line " << std::endl;
        throw std::exception{};
    }

    UserRecord ur = {tokens[0],
        tokens[1],
        tokens[2],
        tokens[3],
        tokens[4]
    };
    return ur;
};

void UserManager::appendToFile(const UserRecord& rec) {
    std::ofstream out{this->users_file, std::ios::app};
    if (!out.is_open()) {
        std::cerr << "UserManager::appendToFile bad file: " << users_file << std::endl;
        return;
    }
}

```

```

        out << rec.username << "," << rec.fullName << "," << rec.email << ","
        << rec.salt << "," << rec.passwordHash << "\n";
    }

void UserManager::rewriteFile() {
    std::ofstream out{this->users_file};
    if (!out.is_open()) {
        std::cerr << "UserManager::rewriteFile bad file: " << users_file << std::endl;
        return;
    }
    for (const UserRecord& r : this->users) {
        out << r.username << ","
            << r.fullName << ","
            << r.email << ","
            << r.salt << ","
            << r.passwordHash << "\n";
    }
}

bool UserManager::exists(std::string& email, std::string& fullName) {
    for (const UserRecord& r : users) { // this->users
        if (r.email == email && r.fullName == fullName) { return true; }
    }
    return false;
};

std::string UserManager::readLine(const std::string& prompt) {
    std::cout << prompt;
    std::string line;
    std::getline(std::cin, line);
    size_t start = line.find_first_not_of(" \t\r\n");
    size_t end = line.find_last_not_of(" \t\r\n");
    if (start == std::string::npos) return "";
    return line.substr(start, end - start + 1);
}

UserRecord* UserManager::findByUsername(const std::string& username) {
    for (UserRecord& r : users) {
        if (r.username == username) return &r;
    }
    return nullptr;
}

void UserManager::ensureFilesExist() {
    std::ifstream check{users_file};
    if (!check.is_open()) {
        std::ofstream create{users_file};
    }
}

```

```
// =====
// ==                                UserWallet.cpp                                ==
// =====

#include "UserWallet.h"
#include "CSVReader.h"
#include <fstream>
#include <iostream>
#include <iomanip>

UserWallet::UserWallet(const std::string& filename) : filename(filename) {
    ensureFilesExist();
    load();
}

void UserWallet::ensureFilesExist() {
    std::ifstream check{filename};
    if (!check.is_open()) {
        std::ofstream create{filename};
    }
}

void UserWallet::load() {
    balances.clear();
    std::ifstream file{filename};
    if (!file.is_open()) return;
    std::string line;
    while (std::getline(file, line)) {
        if (line.empty()) continue;
        std::vector<std::string> tokens = CSVReader::tokenise(line, ',');
        if (tokens.size() != 2) continue;
        try {
            balances[tokens[0]] = std::stod(tokens[1]);
        } catch (const std::exception& e) {
        }
    }
}

void UserWallet::save() {
    std::ofstream file{filename};
    if (!file.is_open()) {
        std::cerr << "UserWallet::save bad file: " << filename << std::endl;
        return;
    }
    for (const auto& entry : balances) {
        file << entry.first << ","
            << std::fixed << std::setprecision(2) << entry.second << "\n";
    }
}

void UserWallet::createAccount(std::string username, double initialBalance) {
    balances[username] = initialBalance;
    save();
}

bool UserWallet::hasAccount(std::string username) {
    return balances.find(username) != balances.end();
}

double UserWallet::getBalance(std::string username) {
    if (hasAccount(username)) return balances[username];
    return 0.0;
}

void UserWallet::deposit(std::string username, double amount) {
    balances[username] += amount;
    save();
}
```

```

}

bool UserWallet::withdraw(std::string username, double amount) {
    if (getBalance(username) < amount) return false;    // insufficient -> caller decides
message
    balances[username] -= amount;
    save();
    return true;
}

void UserWallet::depositFlow(std::string username) {
    double amount = readAmount("Deposit amount: ");
    if (amount <= 0) { std::cout << "Deposit cancelled." << std::endl; return; }
    deposit(username, amount);
    std::cout << "Deposited. New balance: " << getBalance(username) << std::endl;
}

void UserWallet::withdrawFlow(std::string username) {
    double amount = readAmount("Withdraw amount: ");
    if (amount <= 0) { std::cout << "Withdrawal cancelled." << std::endl; return; }
    if (!withdraw(username, amount)) {
        std::cout << "Insufficient funds. Balance unchanged: "
            << getBalance(username) << std::endl;
    } else {
        std::cout << "Withdrawn. New balance: " << getBalance(username) << std::endl;
    }
}

double UserWallet::readAmount(const std::string& prompt) {
    while (true) {
        std::cout << prompt;
        std::string line;
        if (!std::getline(std::cin, line) || line.empty()) return -1.0;    // cancel / EOF
        try {
            double value = std::stod(line);
            if (value > 0) return value;
            std::cout << "Amount must be greater than zero." << std::endl;
        } catch (const std::exception& e) {
            std::cout << "Please enter a valid number." << std::endl;
        }
    }
}

```



```

// =====
// ==                               Wallet.cpp                               ==
// =====

#include "Wallet.h"
#include <iostream>
#include "CSVReader.h"

Wallet::Wallet()
{

}

void Wallet::insertCurrency(std::string type, double amount)
{
    double balance;
    if (amount < 0)
    {
        throw std::exception{};
    }
    if (currencies.count(type) == 0) // not there yet
    {
        balance = 0;
    }
    else { // is there
        balance = currencies[type];
    }
    balance += amount;
    currencies[type] = balance;
}

bool Wallet::removeCurrency(std::string type, double amount)
{
    if (amount < 0)
    {
        return false;
    }
    if (currencies.count(type) == 0) // not there yet
    {
        //std::cout << "No currency for " << type << std::endl;
        return false;
    }
    else { // is there - do we have enough
        if (containsCurrency(type, amount)) // we have enough
        {
            //std::cout << "Removing " << type << ": " << amount << std::endl;
            currencies[type] -= amount;
            return true;
        }
        else // they have it but not enough.
            return false;
    }
}

bool Wallet::containsCurrency(std::string type, double amount)
{
    if (currencies.count(type) == 0) // not there yet
        return false;
    else
        return currencies[type] >= amount;
}

std::string Wallet::toString()
{
    std::string s;
    for (std::pair<std::string, double> pair : currencies)
    {
        std::string currency = pair.first;
        double amount = pair.second;
    }
}

```

```

        s += currency + " : " + std::to_string(amount) + "\n";
    }
    return s;
}

bool Wallet::canFulfillOrder(OrderBookEntry order)
{
    std::vector<std::string> currs = CSVReader::tokenise(order.product, '/');
    // ask
    if (order.orderType == OrderBookType::ask)
    {
        double amount = order.amount;
        std::string currency = currs[0];
        std::cout << "Wallet::canFulfillOrder " << currency << " : " << amount << std::endl;

        return containsCurrency(currency, amount);
    }
    // bid
    if (order.orderType == OrderBookType::bid)
    {
        double amount = order.amount * order.price;
        std::string currency = currs[1];
        std::cout << "Wallet::canFulfillOrder " << currency << " : " << amount << std::endl;

        return containsCurrency(currency, amount);
    }

    return false;
}

void Wallet::processSale(OrderBookEntry& sale)
{
    std::vector<std::string> currs = CSVReader::tokenise(sale.product, '/');
    // ask
    if (sale.orderType == OrderBookType::asksale)
    {
        double outgoingAmount = sale.amount;
        std::string outgoingCurrency = currs[0];
        double incomingAmount = sale.amount * sale.price;
        std::string incomingCurrency = currs[1];

        currencies[incomingCurrency] += incomingAmount;
        currencies[outgoingCurrency] -= outgoingAmount;
    }
    // bid
    if (sale.orderType == OrderBookType::bidsale)
    {
        double incomingAmount = sale.amount;
        std::string incomingCurrency = currs[0];
        double outgoingAmount = sale.amount * sale.price;
        std::string outgoingCurrency = currs[1];

        currencies[incomingCurrency] += incomingAmount;
        currencies[outgoingCurrency] -= outgoingAmount;
    }
}

std::ostream& operator<<(std::ostream& os, Wallet& wallet)
{
    os << wallet.toString();
    return os;
}

```

```

// =====
// ==                                batch_sim.cpp                                ==
// =====

#include "OrderBook.h"
#include "UserWallet.h"
#include "UserManager.h"
#include "TransactionLog.h"
#include "Simulator.h"
#include <iostream>
#include <vector>
#include <string>

/** Custom script to create and run many user simulations at once to populate csv data. *
/
int main() {
    std::cout << "Starting automated account creation and trading simulation..." << std::
endl;

    // Initialize components
    OrderBook orderBook{"./20200317.csv"};
    UserWallet userWallet{"data/USERS_WALLET.CSV"};
    UserManager userManager{userWallet};
    TransactionLog txLog{"data/USERS_TRADING.CSV"};
    Simulator simulator{orderBook, userWallet, txLog};

    // Data for 5 accounts
    struct UserData {
        std::string name;
        std::string email;
        std::string password;
    };

    std::vector<UserData> newUsers = {
        {"Alice Trader", "alice@example.com", "pass123"},
        {"Bob Investor", "bob@example.com", "secure456"},
        {"Charlie Scalper", "charlie@example.com", "charlie789"},
        {"David Holder", "david@example.com", "dave_pass"},
        {"Eve Bot", "eve@example.com", "eve_secret"}
    };

    std::vector<std::string> registeredUsernames;

    for (const auto& user : newUsers) {
        std::cout << "\nRegistering: " << user.name << " (" << user.email << ")" << std::
endl;
        if (userManager.registerUserProgrammatic(user.name, user.email, user.password)) {
            std::string username = userManager.getLatestUsername();
            registeredUsernames.push_back(username);
            std::cout << "Successfully registered " << user.name << " with username: " <<
username << std::endl;
        } else {
            std::cout << "Failed to register " << user.name << " (maybe already exists?)"
<< std::endl;
            // If already exists, we might want to find the username if we wanted to run
sim anyway,
            // but let's assume we want fresh ones or we can just proceed if it's already
there.
        }
    }

    std::cout << "\nRunning trading simulation for all accounts..." << std::endl;
    for (const auto& username : registeredUsernames) {
        std::cout << "Simulating for user: " << username << std::endl;
        simulator.run(username);
    }

    std::cout << "\nBatch process complete!" << std::endl;
    return 0;
}

```

```
// =====  
// ==                                main.cpp                                ==  
// =====  
  
// #include "Wallet.h"  
// #include <iostream>  
#include "MerkelMain.h"  
  
/** Main entry point. Start the app. */  
int main()  
{  
    MerkelMain app{};  
    app.init();  
  
}
```